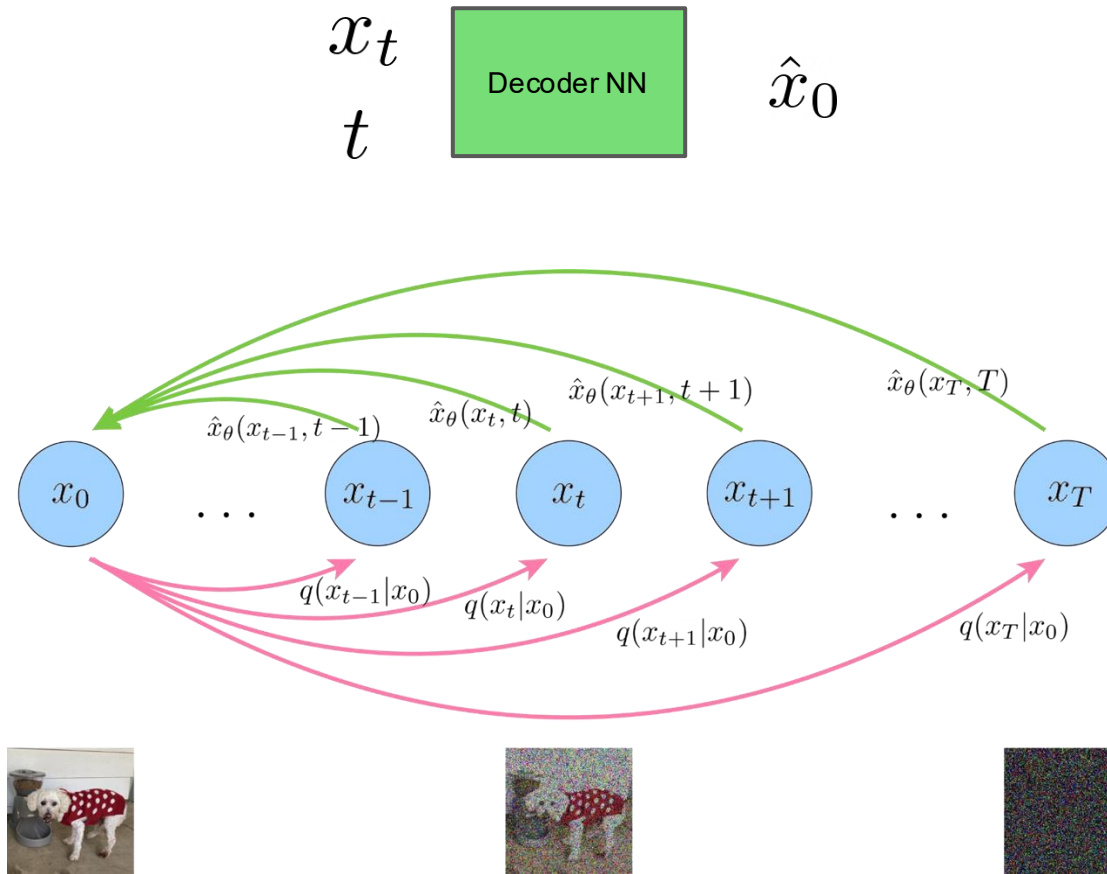


Recall: diffusion process



Diffusion Models: A Summary

- One NN that predicts a clean image from a noisy version of the image



Pseudocode

Algorithm 1 Training

```
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$  Sample image
3:    $t \sim \text{Uniform}(1, \dots, T)$  Sample step
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  Sample noise
5:   Take gradient descent step on
6:      $\nabla_{\theta} \|\mathbf{x}_0 - \hat{\mathbf{x}}_{\theta}(\mathbf{x}_0 + \alpha_t \epsilon, t)\|^2$ 
7: until converged
```

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$ :
3:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\epsilon = 0$ 
4:    $\mathbf{x}_{t-1} = \hat{\mathbf{x}}_{\theta}(\mathbf{x}_t, t) + \alpha_{t-1} \epsilon$ 
5: end for Predict the initial image
6: return  $\mathbf{x}_0$ 
```

Image 🖼️ and Noise 🔊 relation

- Recall that our objective is to predict $\hat{x}_\theta(x_t, t) \approx x_0$
- For arbitrary $x_t \sim q(x_t | x_0)$, we can rewrite it as $x_t = x_0 + \alpha_t \epsilon_0$
- Thus, predicting ϵ_0 determines x_0 and vice-versa

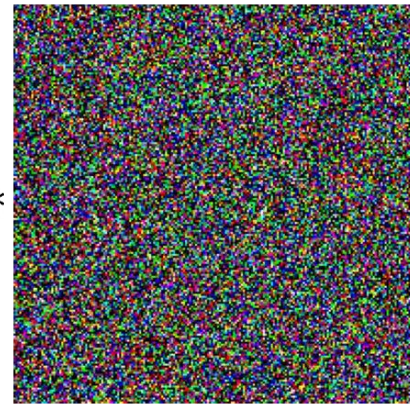


$$x_t \sim q(x_t | x_0)$$

=



+ $\alpha_t *$



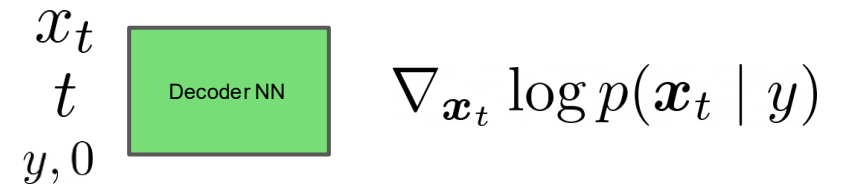
Classifier-Free Guidance

$$\nabla \log p(\mathbf{x}_t \mid y) = \underbrace{\gamma \nabla \log p(\mathbf{x}_t \mid y)}_{\text{conditional score}} + \underbrace{(1 - \gamma) \nabla \log p(\mathbf{x}_t)}_{\text{unconditional score}}$$

- What happens if γ is 1?
- What happens if γ is larger than 1?
- How many diffusion models do we need to train?

$$s_{\theta}(\mathbf{x}_t, t, y) \approx \nabla \log p(\mathbf{x}_t \mid y)$$

$$s_{\theta}(\mathbf{x}_t, t) \approx \nabla \log p(\mathbf{x}_t)$$



Just one - as long as we train it with dropout on the conditioning info!

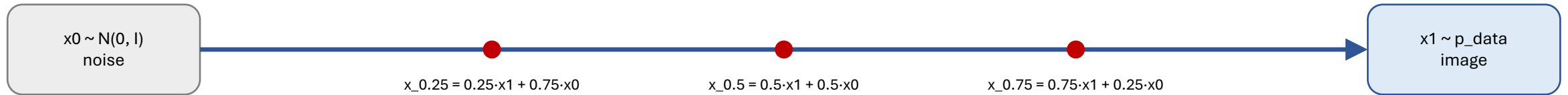
Latent Diffusion Models

Instead of working in the pixel-level:

1. Encode with a compression model (VAE)
2. Perform diffusion process in the latent space
3. Reconstruct the image from the latent

Flow matching in one picture

velocity along the line is constant: $v = x_1 - x_0 \rightarrow$ regress $v_\theta(x_t, t)$ onto $x_1 - x_0$



sample: $x \leftarrow x + \Delta t \cdot v_\theta(x, t)$, from $t = 0$ to 1

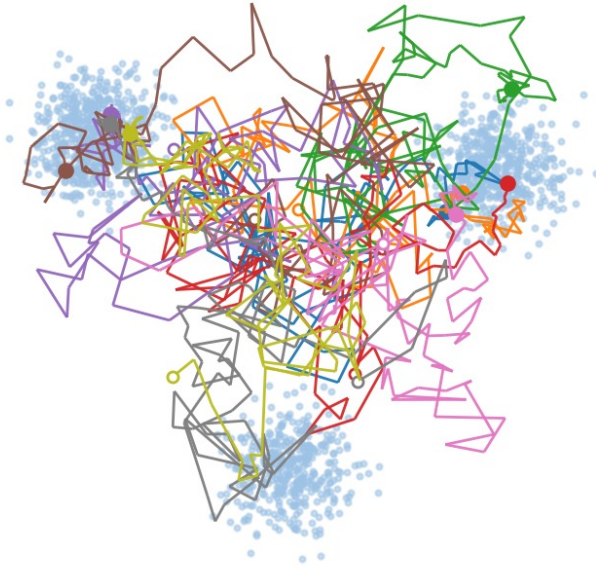
Draw a noise sample x_0 and a data sample x_1 ; interpolate $x_t = t \cdot x_1 + (1-t) \cdot x_0$

The velocity along that line is constant: $x_1 - x_0$ — a regression target with no noise schedule, no ELBO

Train: $L(\theta) = E \|v_\theta(x_t, t) - (x_1 - x_0)\|^2$; sample: Euler steps of the ODE $dx/dt = v_\theta(x, t)$ from $t = 0$ to 1

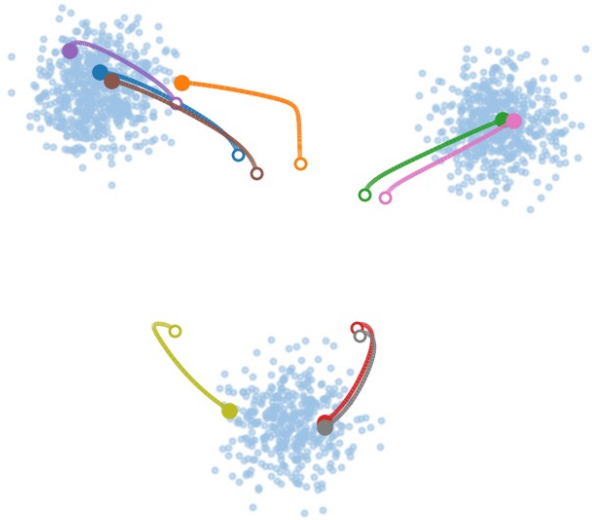
Why straight paths? Three samplers on the same toy data

Diffusion, stochastic sampling (DDPM)
1000 steps, paths wander



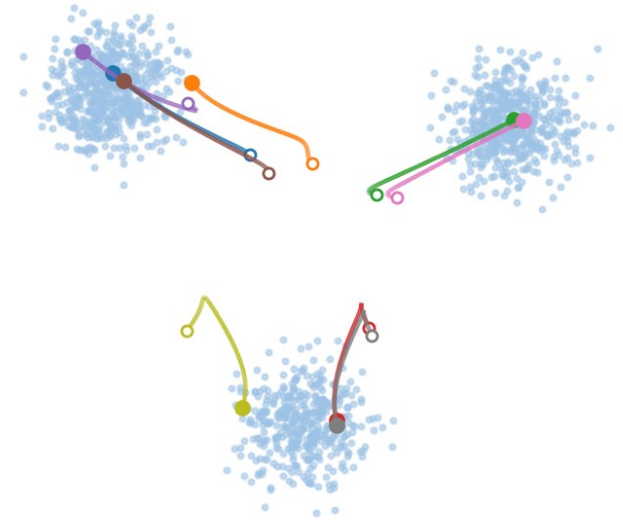
○ noise sample ● generated sample

Diffusion, probability-flow ODE (DDIM)
deterministic but curved



○ noise sample ● generated sample

Flow matching, linear path
nearly straight



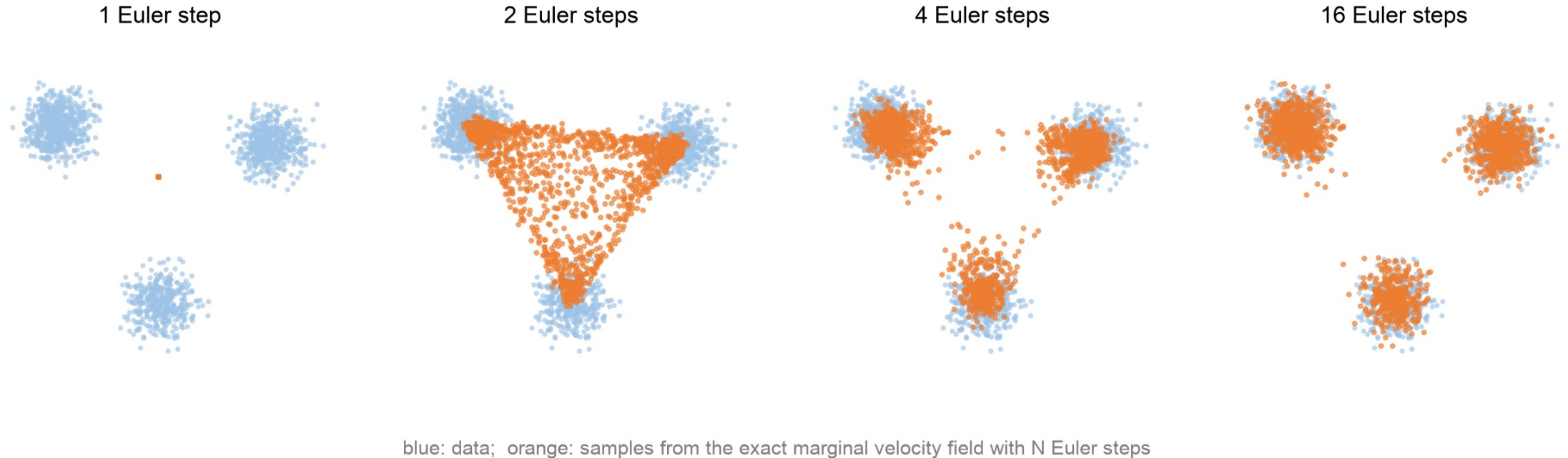
○ noise sample ● generated sample

Left: DDPM ancestral sampling — 1000 stochastic steps, paths wander before landing

Middle: the probability-flow ODE of the same diffusion (DDIM) — deterministic but curved

Right: flow matching with the linear path — nearly straight, so a few Euler steps suffice

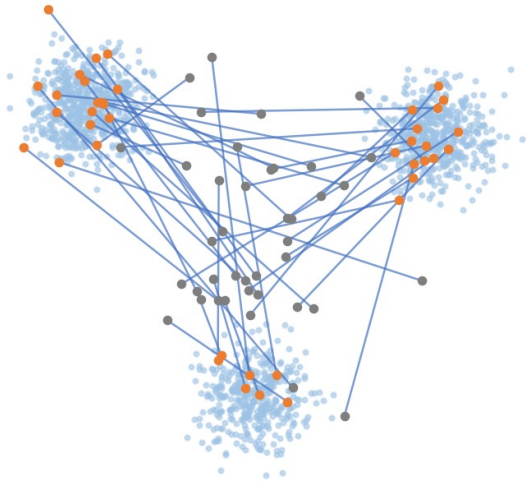
Few steps work only if the marginal field is nearly straight



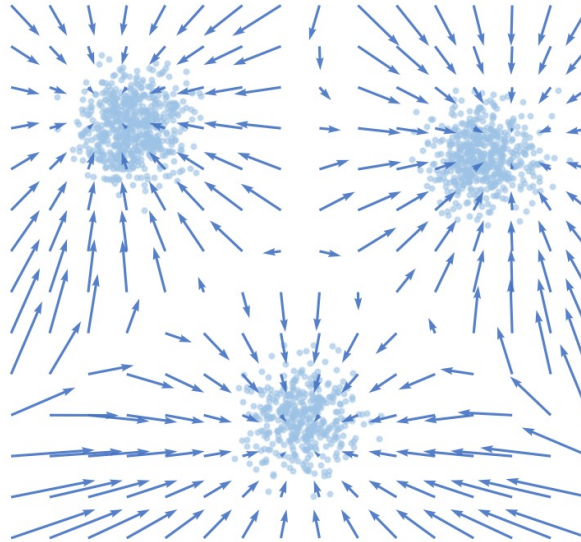
Even with the *exact* marginal velocity, one Euler step lands on the conditional mean, not on the data
Where conditional paths cross, the marginal field curves — this is what reflow, distillation and one-step models fix

Conditional targets vs. the marginal field (toy data)

Conditional paths: one straight line per (x_0, x_1) pair
(independent coupling: lines cross)



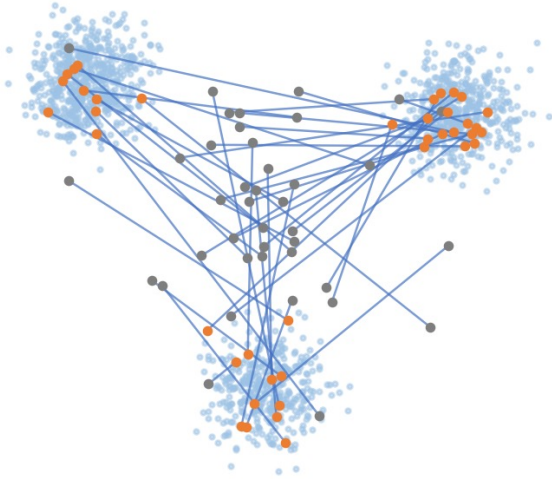
Marginal velocity $u_t(x) = E[x_1 - x_0 \mid x_t = x]$ ($t = 0.5$)
— the average the network must learn (x_0 : noise, x_1 : data)



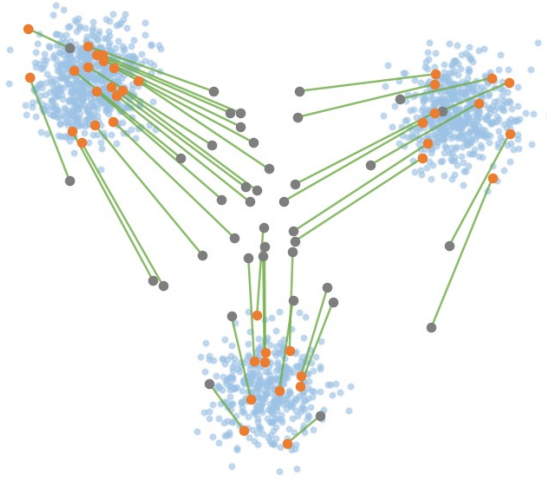
- Left: each training pair contributes one straight line; with independent pairing the lines cross
- Right: the marginal velocity $u_t(x) = E[x_1 - x_0 \mid x_t = x]$ the network actually learns ($t = 0.5$, computed exactly)

Rectified flow and reflow

Independent coupling (x_0, x_1): crossing lines
→ marginal flow must curve



Reflow coupling ($x_0, \text{Flow}(x_0)$): no crossings
→ retrain on these pairs: straight, 1–2 steps



- Three groups, one algorithm (ICLR 2023): flow matching (Lipman), rectified flow (Liu), stochastic interpolants (Albergo)
- **Reflow**: generate pairs ($x_0, \text{Flow}(x_0)$) with the trained model and retrain on them — pairs no longer cross, the field straightens, 1–2 Euler steps become accurate
- Used by InstaFlow and as the 'rectified' in Stable Diffusion 3

Diffusion is the Gaussian-path special case

- Choose the Gaussian path $x_t = \alpha_t x_1 + \sigma_t x_0$ instead of the straight line: flow matching becomes DDPM/score matching with a different parameterization
- Velocity, score, ε -prediction and x -prediction are linear functions of each other — 'two sides of the same coin' (Gao et al., DeepMind blog, Dec 2024)
- Deterministic diffusion sampling (DDIM, probability-flow ODE) *is* a flow; what is new is the straight path, the simpler loss and the freedom to choose the timestep weighting

Guidance carries over unchanged

- Train with the condition randomly dropped; sample with $v = (1-w) \cdot v(x|\emptyset) + w \cdot v(x|y)$ — classifier-free guidance (Lecture 10, slides 59–62) as a combination of velocities
- Latent generation (Lecture 10, slide 63) is unchanged too: run the flow in the VAE latent space
- Almost every deployed image/video model uses guidance (SD3: $w \approx 4$); it trades diversity for prompt adherence

From many steps to one: distillation, consistency, flow maps

MeanFlow / flow map: learn the average velocity between any two times (r, t)



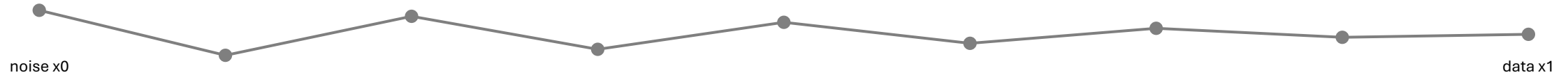
consistency model: any point on the trajectory $\rightarrow$ its end point (one step)



progressive distillation: one student step = two teacher steps; repeat



teacher: an ODE solver takes many small steps along a curved trajectory



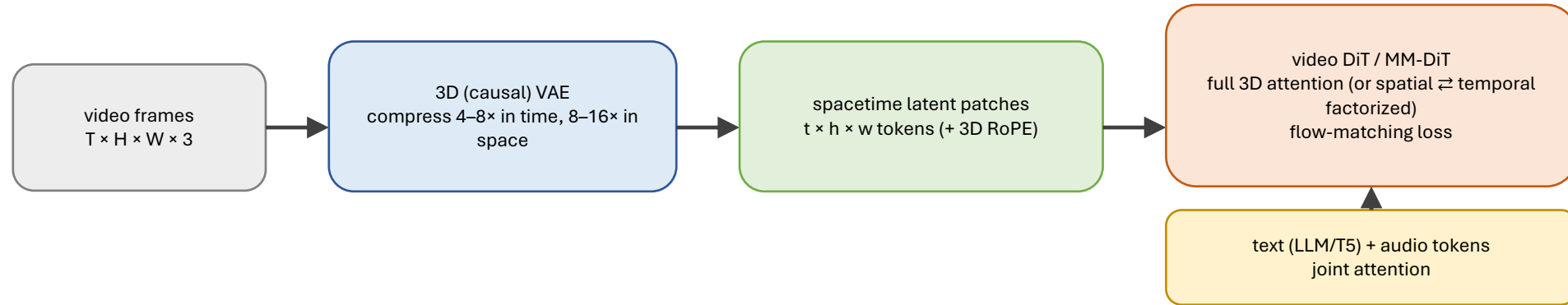
Solvers (Heun, DPM-Solver): 10–50 steps with no retraining

Progressive distillation (Salimans & Ho, 2022): a student matches two teacher steps with one; repeat

Consistency models (Song et al., 2023): map any point on a trajectory to its end point; train by distillation or from scratch

Flow maps (2025): learn the solution operator $\Phi(x, r, t)$ directly; **MeanFlow** learns the average velocity over (r, t) from scratch

Video generators are latent flow-matching transformers with a time axis



Sora (Feb 2024), Movie Gen (Oct 2024, 30B), Wan 2.x, LTX-2, Veo 3 and Sora 2 (2025) all follow this template; older: 3D U-Nets with factorized attention

A 3D VAE compresses time and space; spacetime patches become tokens with 3D positional encoding
Full 3D attention (or spatial ⇌ temporal factorized attention in older 3D U-Nets); text and now audio tokens attend jointly with video tokens
Sampling and RL tricks carry over from images: few-step distillation, CFG, reward fine-tuning